

Table of Contents

Documentazione del Sistema di Routing API - Componente Router.....	2
1 Router Class con Pattern Command.....	2
1.1 Panoramica.....	2
1.2 Architettura.....	2
1.3 File di Configurazione Routes.....	2
1.4 Flusso del Routing.....	3
1.5 Gestione degli Errori.....	3
1.6 Metodi Principali.....	3
Costruttore.....	3
Verifica Route.....	4
Dispatch Route.....	4
Gestione Richiesta Completa.....	4
Estrazione Route da URI.....	4
1.7 Pattern Command Implementato.....	4
1.8 Messaggi di Errore.....	4
1.9 Esempi di Utilizzo.....	5
Esempio 1: Utilizzo Base.....	5
Esempio 2: Dispatch Manuale.....	5
Esempio 3: Testing delle Route.....	5
1.10 Integrazione con il Sistema Completo.....	5
1.11 Vantaggi del Design.....	5
Separazione delle Preoccupazioni.....	5
Estensibilita'.....	6
Testabilita'.....	6
Gestione Errori Uniforme.....	6
1.12 Dettagli Implementazione.....	6
Caricamento Route da File.....	6
Gestione URI con Query Parameters.....	6
Validazione a Multipli Livelli.....	7
1.13 Strategia di Testing.....	7
1.14 Sezioni di Test Implementate.....	7
1.15 Considerazioni di Sicurezza.....	9
1.16 Considerazioni sulle Performance.....	9
1.17 Punti di Estensione.....	9
1.18 Esecuzione dei Test.....	9
1.19 Risultati Attesi dai Test.....	10
1.20 Conclusione.....	11

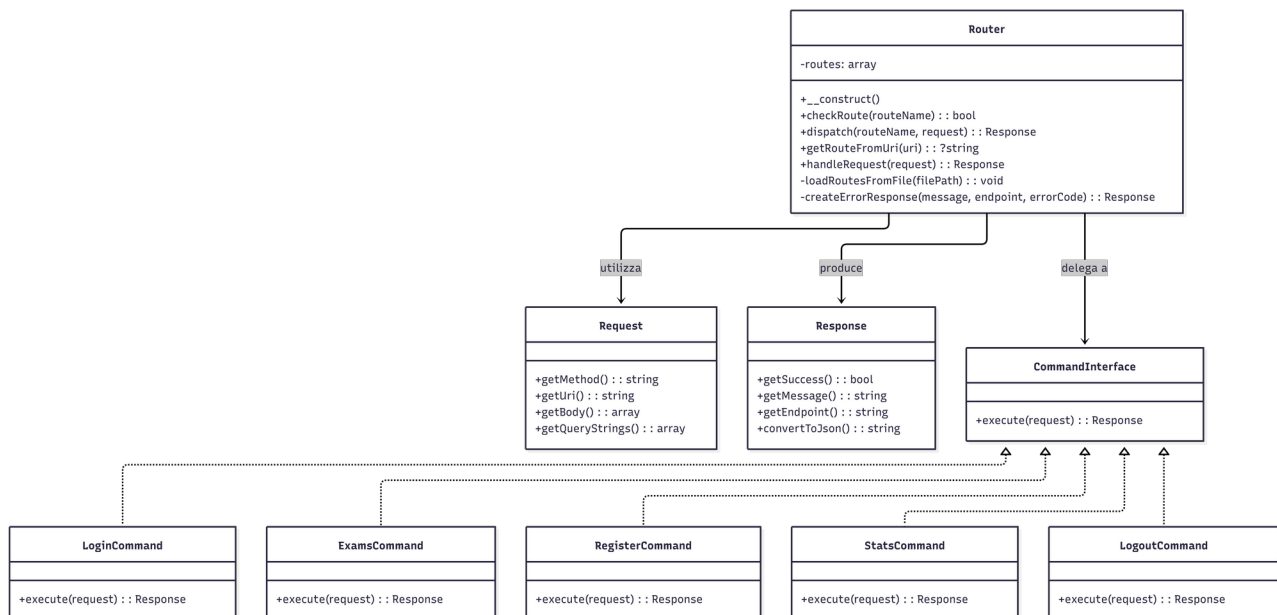
Documentazione del Sistema di Routing API - Componente Router

1 Router Class con Pattern Command

1.1 Panoramica

Il componente **Router** implementa il **Design Pattern Command** per gestire il routing delle richieste API. Questo pattern incapsula ogni endpoint in un oggetto comando separato, permettendo una gestione flessibile e modulare delle richieste HTTP.

1.2 Architettura

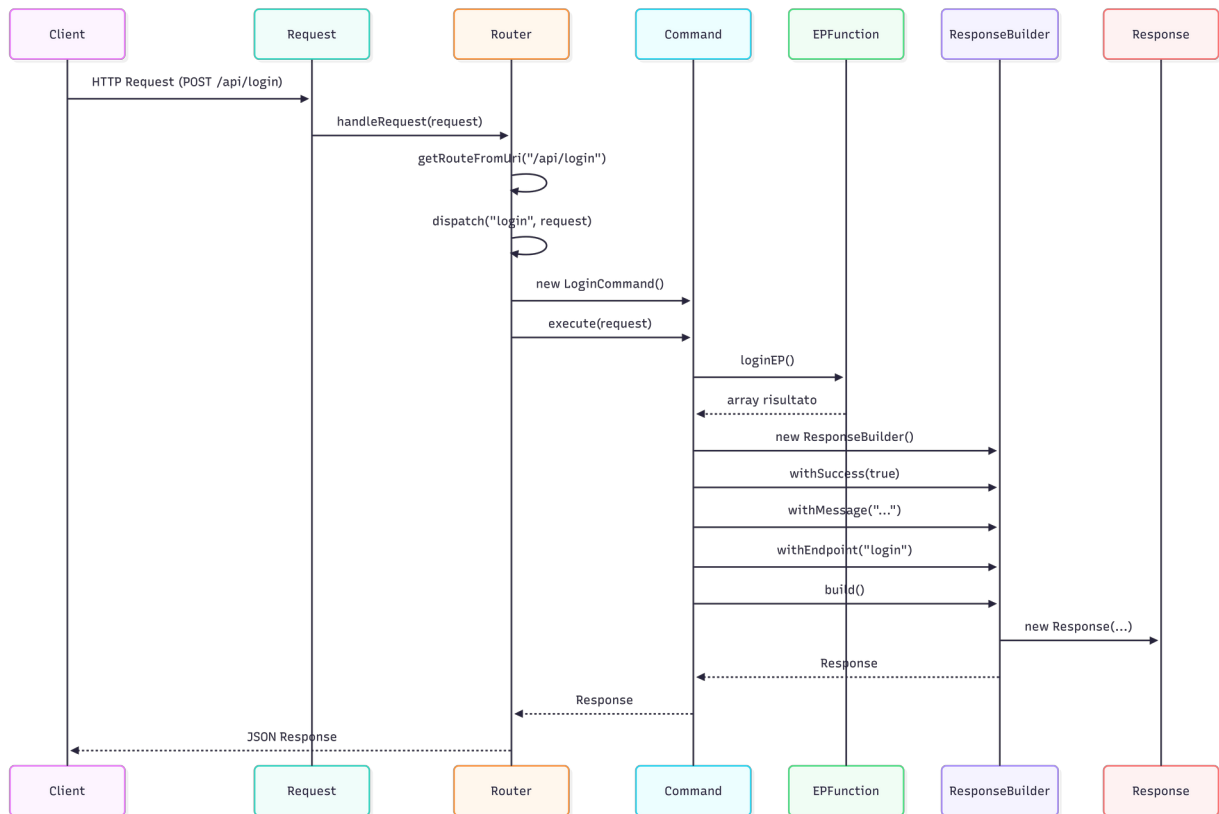


1.3 File di Configurazione Routes

Il router carica le route da un file di configurazione (`endpoints.txt`):

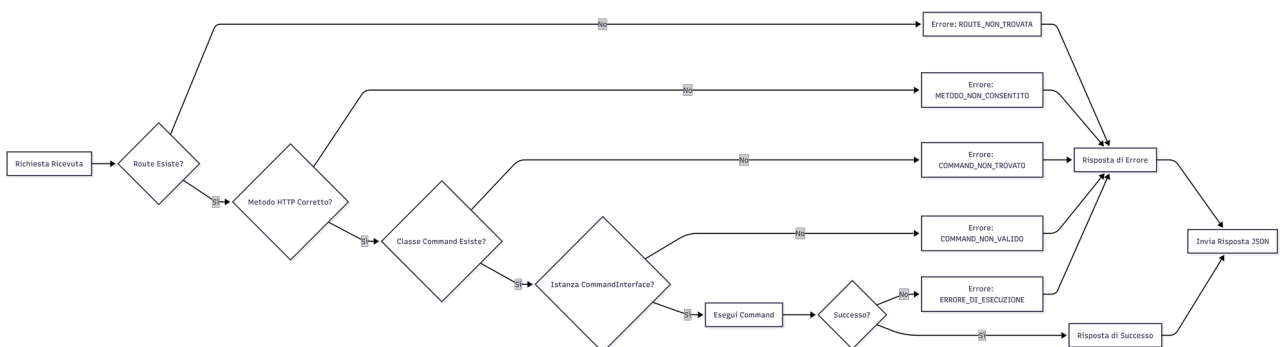
```
# Formato: nome_route METODO_HTTP NomeClasseCommandBase
login POST Login
register POST Register
exams GET Exams
stats GET Stats
logout POST Logout
```

1.4 Flusso del Routing



1.5 Gestione degli Errori

Il router include una gestione errori completa:



1.6 Metodi Principali

Costruttore

```
public function __construct()
```

Carica automaticamente le route dal file `endpoints.txt`.

Verifica Route

```
public function checkRoute(string $routeName): bool
```

Verifica se una route esiste nel router.

Dispatch Route

```
public function dispatch(string $routeName, Request $request): Response
```

Esegue il comando associato alla route.

Gestione Richiesta Completa

```
public function handleRequest(Request $request): Response
```

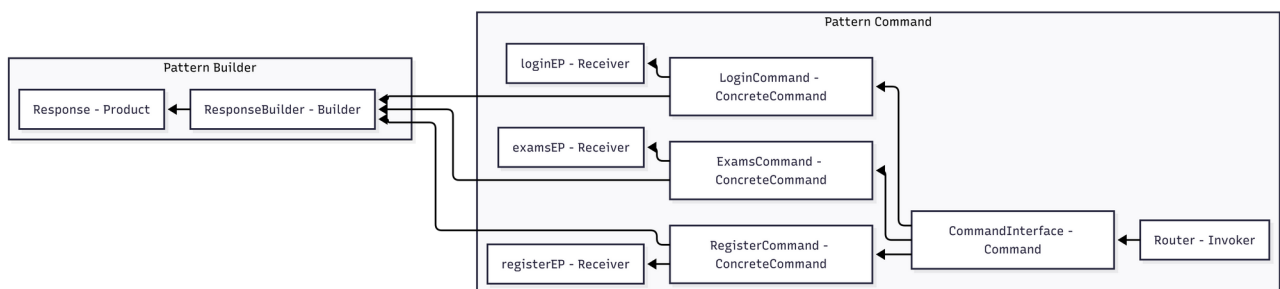
Gestisce una richiesta completa dall'URI al risultato.

Estrazione Route da URI

```
public function getRouteFromUri(string $uri): ?string
```

Converte un URI in un nome di route (gestisce query parameters).

1.7 Pattern Command Implementato



1.8 Messaggi di Errore

Codice Errore	Messaggio	Causa
ROUTE_NON_TROVATA	"Route non trovata"	La route richiesta non esiste
METODO_NON_CONSENTITO	"Metodo non consentito"	Metodo HTTP errato per la route
COMMAND_NON_TROVATO	"Command non trovato"	Classe Command non esiste
COMMAND_NON_VALIDO	"Command non valido"	Command non implementa CommandInterface
ERRORE_DI_ESECUZIONE	"Esecuzione fallita"	Errore durante l'esecuzione del command

1.9 Esempi di Utilizzo

Esempio 1: Utilizzo Base

```
// Crea router e gestisci richiesta corrente
$router = new Router();
$request = Request::createBaseRequest();
$response = $router->handleRequest($request);

// Invia risposta
header('Content-Type: application/json');
echo $response->convertToJson();
```

Esempio 2: Dispatch Manuale

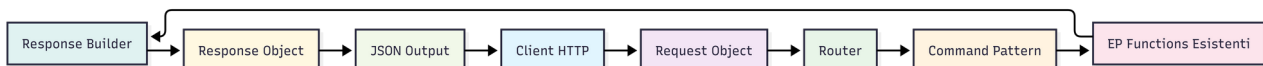
```
// Dispatch manuale di una route specifica
$router = new Router();
$request = new Request('GET', '/exams', ['Authorization' => 'Bearer...']);
$response = $router->dispatch('exams', $request);
```

Esempio 3: Testing delle Route

```
// Verifica esistenza route
$router = new Router();
if ($router->checkRoute('login')) {
    echo "Route login esiste";
}

// Estrazione route da URI
$routeName = $router->getRouteFromUri('/stats?anno=2024');
// Restituisce: 'stats'
```

1.10 Integrazione con il Sistema Completo



1.11 Vantaggi del Design

Separazione delle Preoccupazioni

- **Router:** Solo logica di routing
- **Commands:** Logica business specifica per endpoint
- **EP Functions:** Codice esistente rimane intatto
- **Response Builder:** Formattazione risposte

Estensibilit 

```
// Aggiungere nuove route e' semplice:  
// 1. Creare nuovo command class  
// 2. Aggiungere a endpoints.txt  
// 3. Nessuna modifica al Router necessario
```

Testabilit 

```
// Ogni command puo' essere testato isolatamente  
$command = new LoginCommand();  
$testRequest = new Request('POST', '/login', [], $credenziali);  
$response = $command->execute($testRequest);  
Assertion::assertTrue($response->getSuccess());
```

Gestione Errori Uniforme

```
// Tutti gli errori seguono lo stesso formato  
private function createErrorResponse(  
    string $message,  
    string $endpoint,  
    string $errorCode  
): Response {  
    return (new ResponseBuilder())  
        ->withSuccess(false)  
        ->withMessage($message)  
        ->withEndpoint($endpoint)  
        ->withErrorCode($errorCode)  
        ->build();  
}
```

1.12 Dettagli Implementazione

Caricamento Route da File

```
private function loadRoutesFromFile(string $filePath): void
```

- Legge file di configurazione
- Salta righe vuote e commenti (#)
- Formato: route metodo command
- Memorizza in array associativo (chiave -> valore)

Gestione URI con Query Parameters

```
public function getRouteFromUri(string $uri): ?string
```

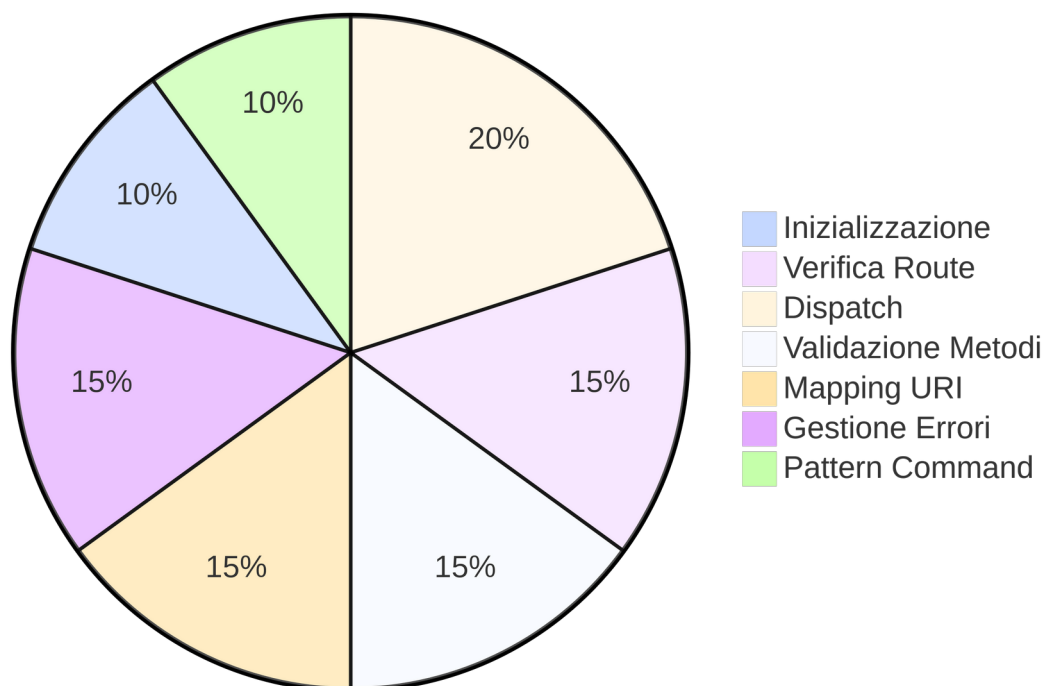
- Rimuove slashes iniziali/finali
- Rimuove query parameters (strtok(\$uri, '?'))
- Cerca corrispondenza esatta con nomi route

Validazione a Multipli Livelli

- 1 **Esistenza Route:** `checkRoute()`
- 2 **Metodo HTTP:** Confronta con metodo richiesto
- 3 **Classe Command:** Verifica esistenza classe
- 4 **Interface Command:** Verifica implementazione `CommandInterface`
- 5 **Esecuzione:** Try-catch per errori di runtime

1.13 Strategia di Testing

Categorie di Test Router



1.14 Sezioni di Test Implementate

Test dell'inizializzazione del Router

- Creazione istanza Router
- Verifica proprietà routes

Test del controllo delle route

- Verifica route esistenti
- Verifica route non esistenti
- Case sensitivity

Test del dispatcher

- Dispatch route login
- Dispatch route exams
- Dispatch route register

Test della validazione dei metodi HTTP

- Metodo errato su login
- Metodo errato su exams
- Metodo corretto

Test della mappatura degli Uri

- URI semplici
- URI con slash
- URI non esistenti
- URI root

Test della gestione degli errori

- Route non esistente
- Command mancante

Test dell'implementazione del design pattern Command

- Esecuzione LoginCommand
- Esecuzione ExamsCommand
- Verifica CommandInterface

Test completo della gestione delle Request

- Gestione login completa
- Gestione exams completa
- Gestione route non esistenti
- Gestione parametri query

1.15 Considerazioni di Sicurezza

- 1 **Validazione Input:** Tutti i parametri validati
- 2 **Metodo HTTP:** Verifica dei metodi consentiti
- 3 **Error Handling:** Messaggi di errore generici (no dettagli interni)
- 4 **Type Safety:** Previene type-injection
- 5 **Immutabilit :** Previene manipolazione request/response

1.16 Considerazioni sulle Performance

- 1 **Caricamento Lazy:** Route caricate solo al bisogno
- 2 **Cache:** Routes memorizzate in array dopo caricamento
- 3 **Ricerca Veloce:** Array associativo per lookup delle route
- 4 **Overhead Minimo:** Delega diretta ai commands

1.17 Punti di Estensione

La classe Router puo' essere estesa per funzionalita' aggiuntive sia nella classe stessa che tramite il design pattern Command ma non ne parleremo in questo documento poiche' non richiesto.

1.18 Esecuzione dei Test

Per eseguire i test Response, una volta lanciata l'applicazione con Docker:

```
docker compose up --force-recreate --remove-orphans -d
```

Bastera' navigare presso la pagina <http://localhost:8080/test/> e cliccare il link [Test Page for Response](#).

1.19 Risultati Attesi dai Test

Ogni test produrrà output simile a:

Test della classe Router con design pattern Command

Test dell'inizializzazione del Router

- Creazione dell'istanza di Router -

PASSED: `assertInstanceOf(Router)`

PASSED: `assertNotNull(Router)`

Test del controllo delle route

- Verifica della route login -

PASSED: `assertTrue(router->checkRoute[login])`

...

Con CSS styling che evidenzia:

- Successo: **Contorno verde**

- Fallimento: **Contorno rosso**

1.20 Conclusione

Il componente Router completa il sistema di routing API

1. **Pattern Command** per gestione endpoint modulare
2. **Integrazione** con codice esistente (EP functions)
3. **Gestione errori** robusta e consistente
4. **Testabilita'** completa di tutte le componenti
5. **Estensibilita'** per nuove funzionalita'

Questo sistema fornisce una base solida e una soluzione pratica per il routing API in ambienti reali.